Heuristiken

Carsten Gips (HSBI)

TL;DR
CSP's mit endlichen Domänen lassen sich mit einer Backtracking-Suche lösen. Dabei gibt es einige Freiheitsgrade: Auswahl der nächsten Variable und Wahl des nächsten Werts. Hier können Heuristiken die Suche beschleunigen.
Zur Wahl der als nächstes zu betrachtenden Variable kann die **Minimum Remaining Values (MRV)**-Heuristik eingesetzt werden: Wähle die Variable mit wenigsten freien Werten. Bei Gleichstand bei der MRV kann man mit der **Gradheuristik** die Variable mit den meisten Constraints zu offenen (noch nicht belegten) Variablen wählen.
Bei der Wahl des Wertes kann die **Least Constraining Value (LCV)**-Heuristik genutzt werden: Wähle den Wert, der für die verbleibenden Variablen die wenigsten Werte ungültig macht.
Während die MRV relativ leicht umzusetzen ist, muss man für die LCV alle Constraints zu den Nachbarn auswerten.
Videos

- VL CSP, Heuristiken

---

1 VARIABLES: Variablen-Sortierung, Welche Variable soll betrachtet werden?

```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VARIABLES: Welche Variable zuerst ausprobieren?

Minimum Remaining Values (MRV): (vgl. (Russell und Norvig 2020, 177))

- Wähle Variable mit wenigsten freien Werten (die am meisten eingeschränkte Variable)

  => reduziert den Verzweigungsgrad

Beispiel:

1. Freie Auswahl, alle haben gleich viele freie Werte (jeweils 3) => wähle A
2. B und C haben nur noch zwei freie Werte => wähle B (oder C)
3. C hat nur noch einen Wert, D noch zwei, der Rest drei => wähle C

2 VARIABLES: Gleichstand bei MRV

```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VARIABLES: Welche Variable zuerst ausprobieren?

Gradheuristik: Erweiterung von *MRV* bei *Gleichstand* (vgl. (Russell und Norvig 2020, 177))

- Wähle Variable mit meisten Constraints auf offene (noch nicht zugewiesene) Variablen

  => reduziert den Verzweigungsgrad in späteren Schritten

Beispiel:

1. MRV: Alle haben gleich viele freie Werte (jeweils 3) => Gradheuristik: B, C und D haben die meisten Verbindungen (Constraints) auf offene Variablen => wähle B (oder C oder D)
2. MRV: A, C und D haben nur noch zwei freie Werte => Gradheuristik: C und D haben je zwei Constraints auf noch offene Variablen => wähle C (oder D)
3. MRV: A und D haben beide nur noch einen Wert => Gradheuristik: D hat die meisten Verbindungen (Constraints) auf offene Variablen => wähle D

3 VALUES: Werte-Sortierung, Welchen Wert soll ich ausprobieren?

```
def BT_Search(assignment, csp):
    if complete(assignment): return assignment

    var = VARIABLES(csp, assignment)

    for value in VALUES(csp, var):
        if consistent(value, var, assignment, csp):
            assignment += {var = value}

            if INFERENCE(csp, assignment, var) != failure:
                result = BT_Search(assignment, csp)
                if result != failure: return result

            assignment -= {var = value}

    return failure
```

Quelle: Eigener Code basierend auf einer Idee nach (Russell und Norvig 2020, S. 176, Fig. 5.5)

VALUES: Welchen Wert zuerst ausprobieren?

Least Constraining Value (LCV): (vgl. (Russell und Norvig 2020, 177))

- Wähle Wert, der für verbleibende Variablen die wenigsten Werte ungültig macht

  => verringert die Wahrscheinlichkeit für Backtracking

Beispiel:

1. Sei A gewählt: Alle Werte machen in den anderen Variablen einen Wert ungültig => freie Wahl des Wertes => wähle beispielsweise rot
2. Sei B gewählt: Alle Werte machen in den anderen Variablen einen Wert ungültig => freie Wahl des Wertes => wähle beispielsweise grün
3. Sei D gewählt: Verbleibende Werte rot und blau
   - Wahl von rot würde für C einen Wert übrig lassen (blau)
   - Wahl von blau würde für C **keinen** Wert übrig lassen => LCV: Wahl von rot!

Hinweis: Diese Heuristik ist in der Praxis sehr aufwändig zu berechnen! Man müsste für jeden Wert die noch offenen Constraints anschauen und berechnen, wie viele Werte damit jeweils ungültig gemacht werden. Die Idee ist aber dennoch interessant, und möglicherweise kann man sie für ein reales Problem so adaptieren, dass bei der Umsetzung nur wenig zusätzlicher Aufwand entsteht.

4 Wrap-Up

- Verbesserung der BT-Suche mit Heuristiken: MRV, Gradheuristik, LCV

5 Zum Nachlesen

- Russell und Norvig (2020): CSP, Backtracking/Heuristiken: Abschnitt 5.3
- Kumar (1992)
- Barták (2001)

Lernziele

- k3: Verbesserung der BT-Suche mit Heuristiken: MRV, Gradheuristik, LCV

Quizzes

- Selbsttest CSP, Heuristiken (ILIAS)

Challenges

Sei $D = \{0, \ldots, 5\}$, und ein Constraintproblem definiert durch

$$\langle \{v_1, v_2, v_3, v_4\}, \{D_{v_1} = D_{v_2} = D_{v_3} = D_{v_4} = D\}, \{c_1, c_2, c_3, c_4\} \rangle$$

mit

- $c_1 = \big((v_1, v_2), \{(x, y) \in D^2 | x + y = 3\}\big)$,
- $c_2 = \big((v_2, v_3), \{(x, y) \in D^2 | x + y \leq 3\}\big)$,
- $c_3 = \big((v_1, v_3), \{(x, y) \in D^2 | x \leq y\}\big)$ und
- $c_4 = \big((v_3, v_4), \{(x, y) \in D^2 | x \neq y\}\big)$.

1. Zeichen Sie den Constraint-Graph.
2. Welche Variable würde bei der Anwendung von *MRV* und *Gradheuristik* im ersten Schritt bei der Suche mit der BT-Search ausgewählt?
3. Geben Sie eine Lösung für das Problem an.

Quellen

Barták, R. 2001. „Theory and Practice of Constraint Propagation“. In *Proceedings of the 3rd Workshop on Constraint Programming for Decision and Control*. Bd. 50.

Kumar, V. 1992. „Algorithms for Constraint Satisfaction Problems: A Survey“. *AI Magazine* 13 (1): 32–44.

Russell, S., und P. Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 5f0b0a3 (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-17)